American Computer Science League

2024 Finals ● Short Problems Solutions ● Senior Division

1. Boolean Algebra

$\overline{\overline{A(B + \overline{C})} \oplus \overline{ABC}}$
$= (\overline{A} + \overline{B + \overline{C}}) \oplus \overline{ABC}$
$= (\overline{A} + \overline{B}\,\overline{\overline{C}}) \oplus \overline{ABC}$
$= (\overline{A} + \overline{B}\,C) \oplus \overline{ABC}$
$= \overline{(\overline{A} + \overline{B}\,C)}\;\overline{ABC} + (\overline{A} + \overline{B}\,C)\;\overline{\overline{ABC}}$
$= (\overline{\overline{A}}\;\overline{\overline{B}\,C})\,(\overline{A} + \overline{B} + \overline{C}) + (\overline{A} + \overline{B}\,C)(ABC)$
$= A(B + \overline{C})(\overline{A} + \overline{B} + \overline{C}) + \overline{A}ABC + \overline{B}CABC$
$= (AB + A\overline{C})(\overline{A} + \overline{B} + \overline{C}) + 0 + 0$
$= A\,B\overline{A} + A\,B\overline{B} + A\,B\overline{C} + A\,\overline{C}\,\overline{A} + A\,\overline{C}\,\overline{B} + A\,\overline{C}\,\overline{C}$
$= 0 + 0 + A\,B\,\overline{C} + 0 + A\,\overline{B}\,\overline{C} + A\overline{C}$
$= A\overline{C}(B + \overline{B} + 1)$
$= A\overline{C}$

To be TRUE $A\overline{C} = 1 \rightarrow A = 1 \wedge \overline{C} = 1$.
Therefore, (1,*,0) makes it TRUE. So there are 2 ordered triples.

B. 2

2. Boolean Algebra

$A \oplus (\overline{B\,C} + \overline{A\,B\,C})$
$= A \oplus (\overline{BC} + \overline{ABC})$
$= A \oplus (\overline{B} + \overline{C} + \overline{A} + \overline{B} + \overline{C})$
$= A \oplus (\overline{A} + \overline{B} + \overline{C})$
$= \overline{A}(\overline{A} + \overline{B} + \overline{C}) + A(\overline{\overline{ABC}})$
$= \overline{A} + \overline{A}\,\overline{B} + \overline{A}\,\overline{C} + ABC$
$= \overline{A}(1 + \overline{B} + \overline{C}) + ABC$
$= \overline{A} + ABC$

B. $\overline{A} + ABC$

3. Bit-String Flicking

(RCIRC-5 11011010) XOR (LSHIFT-3 10101101) AND
11001101 OR 10001010
= 11010110 XOR (01101000 AND 11001101) OR 10001010
= (11010110 XOR 010001000) OR 10001010
= 10011110 OR 10001010
= 10011110

A. 10011110

4. Bit-String Flicking

Let X = abcde
LHS = (RCIRC-1 (LSHIFT-2 X)) OR (LCIRC-3 (RSHIFT-2 X))
= (RCIRC-1 (LSHIFT-2 abcde)) OR
(LCIRC-3 (RSHIFT-2 abcde))
= (RCIRC-1 cde00) OR (LCIRC-3 00abc)
= 0cde0 OR bc00a
= bcdea
RHS = X = abcde
LHS = RHS → bcdea = abcde
Therefore a = b = c = d = e
Therefore X = 00000 and 11111 works.

B. 2

5. Recursive Functions

$f(15,11) = 2*f(15-2,11-3) + 1 = 2 * f(13,8) + 1 = 2 * 23 + 1 = 47$
$f(13,8) = 2*f(13-2,8-3) + 1 = 2 * f(11,5) + 1 = 2 * 11 + 1 = 23$
$f(11,5) = 2*f(11-2,5-3) + 1 = 2 * f(9,2) + 1 = 2 * 5 + 1 = 11$
$f(9,2) = f(2-1,9-3) - 1 = f(1,6) - 1 = 5$
$f(1,6) = 1 * 6 = 6$

E. None of the above

6. Recursive Functions

$f(20) = f(20-4) - 3 = f(16) - 3 = 21 - 3 = 18$
$f(16) = f(16-4) - 3 = f(12) - 3 = 24 - 3 = 21$
$f(12) = f(12-4) - 3 = f(8) - 3 = 27 - 3 = 24$
$f(8) = f(8-3) + 2 = f(5) + 2 = 25 + 2 = 27$
$f(5) = 5^2 = 25$
$f(18) = f(18-4) - 3 = f(14) - 3 = 17 - 3 = 14$
$f(14) = f(14-4) - 3 = f(10) - 3 = 20 - 3 = 17$
$f(10) = f(10-3) + 2 = f(7) + 2 = 18 + 2 = 20$
$f(7) = f(7-3) + 2 = f(4) + 2 = 16 + 2 = 18$
$f(4) = 4^2 = 16$
Therefore, $f(f(20)) = f(18) = 14$

A. 14

7. Digital Electronics

The Boolean expression for the circuit is: $\overline{(A + B) + (\overline{BC} \oplus C)}$
$= \overline{(A + B)}\,\overline{(\overline{BC} \oplus C)}$
$= \overline{A}\,\overline{B}\,(\overline{\overline{BC}} \oplus C)$
$= \overline{A}\,\overline{B}\,(BC\,\overline{C} + \overline{BC}\,C)$
$= \overline{A}\,\overline{B}\,B\,C\,\overline{C} + \overline{A}\,\overline{B}(\overline{B} + \overline{C})C$
$= 0 + \overline{A}\,\overline{B}\,C + 0$

D. $\overline{A}\,\overline{B}\,C$.

8. Digital Electronics

The Boolean expression that represents the digital circuit is:
$\overline{((A + B)(\overline{B \oplus C}))\ \overline{C}}$
$= \overline{\overline{A + B} + \overline{\overline{B \oplus C}} + \overline{C}}$
$= \overline{A}\,\overline{B} + (\overline{B}C + B\overline{C}) + C$
If $\overline{A}\,\overline{B} + \overline{B}C + B\overline{C} + C = 0$, then $C = 0$.
So $\overline{A}\,\overline{B} + \overline{B} * 0 + B * 1 + 0 = \overline{A}\,\overline{B} + B = 0$. Then $B = 0$.
Then $\overline{A}\,0 + 0 = \overline{A} * 1 = \overline{A}$. If $\overline{A} = 0$, then $A = 1$.
Therefore (1,0,0) makes it FALSE.

A. 1

9. Prefix-Infix-Postfix

s 5 3 4 * 6 7 + # = in infix is: s = (5 +12 + 13)/2 = 15
A s s 5 − * s 6 2 * − s 5 2 * 3 + − * * \/ = in infix is:
A = \/ ((15 * (15-5)) * ((15-12) * (15-13)) = \/ (15*10*3*2)
= \/ 900 = 30.
The square root is 30 which is the area for a 5-12-13 right triangle using Heron's formula.

D. 30

10. Prefix-Infix-Postfix

2 4 8 2 − * 5 2 ^ / − 8 4 / 2 / 8 * +
= 2 4 (8 2 −) * (5 2 ^) / − (8 4 /) 2 / 8 * +
= 2 [4 (− 8 2) *] (^ 5 2) / − [(/ 8 4) 2 /] 8 * +
= 2 ([* 4 − 8 2] (^ 5 2) /) − ([/ / 8 4 2] 8 *) +
= 2 (/ * 4 − 8 2 ^ 5 2) − (* / / 8 4 2 8) +
= [2 (/ * 4 − 8 2 ^ 5 2) −] (* / / 8 4 2 8) +
= ([− 2 / * 4 − 8 2 ^ 5 2] (* / / 8 4 2 8) +)
= + − 2 / * 4 − 8 2 ^ 5 2 * / / 8 4 2 8

B. +−2/*4−82^52*//8428

11. Computer Number Systems

A_{16} * ($42_8 - F_{16} + 25_{10}$) / 16_{16} can be easily converted to base 10 to do the calculations: 10 * (34 - 15 + 25) / 22 = 10 * 44 / 22 = 20_{10}. That is 16+4 so in hexadecimal, the value is 14_{16}.

A. 14

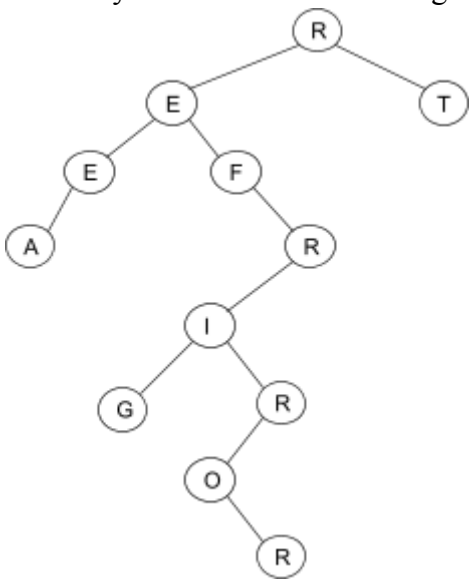
12. Computer Number Systems

When raising 3 to increasing powers in base 10, the results are: 3^1 = 3, 3^2 = 9, 3^3 = 27, 3^4 = 81, 3^5 = 243, 3^6 = 729, etc. The final digit is how much greater each number is than the closest power of 8. That creates the pattern 3, 1, 3, 1, 3, 1 which means that all odd powers end in a 3 and all even powers end in a 1.

B. 3

13. Data Structures

The binary search tree for the string REFRIGERATOR is:



To find the postorder traversal, you must do left-right-root. That means AEGRORIRFETR.

C. AEGRORIRFETR

14. Data Structures

This sequence of PUSH and POP operations on a stack will push the numbers and when an operator is used, it will pop the previous 2 elements on the stack, perform the operation in reverse, and push the result. Therefore, this results in 5 7 * 9 3 / 2 + / 2 − in postfix.

B. 5 7 * 9 3 / 2 + / 2 −

15. Graph Theory

To find all of the paths of length 3, take the original adjacency matrix [M] to the 3rd power so $[M]^3 = [M]^2 * [M]$ as follows:

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}^2 = \begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 2 & 0 & 0 & 2 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix} * [M] = \begin{bmatrix} 2 & 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 2 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{bmatrix}$$

The total number of paths of length 3 is 2+2+2+2+1+1+1+1=12.

D. 12

16. Graph Theory By inspection, there are 10 of them: CBC, CBAC, CBADC, CBEDC, CFDC, CFDAC, CFDABC, CBEDAC, CBAEDC, and CBAFDC. The total number is 17, but ABA, ADA, ABEDA, AEDA, AFDA, DED, and DFD do not include vertex C.	B. 10
17. What Does This Program Do? If p=2 or p=4, the results are the same. In fact, these produce the perfect numbers 28 and 496. If p = 2, 2^2 * (2^3-1) = 4*7 = 28 as does 1+...+(2^3-1)=28, and 1^3+3^3=28 If p = 3, 2^3 * (2^4-1) = 8*15 = 120 as does 1+...+(2^4-1)=120, but 1^3+3^3+5^3=225 If p=4, 2^4 * (2^5-1) = 16*31 = 496 as does 1+2+...+29+(2^5-1)=496 and 1^3 +3^3+5^3+7^3=496	B. 2, 4
18. LISP (SETQ G '(c (a) (a c))) (SETQ H '((a b (c a)) (a (b)))) (CDADR (REVERSE (CONS G (CDDAR H)))) (CDDAR H) = (CDDR ‘(a b (c a))) = (CDR ‘(b (c a))) = ((ca)) (CONS G ‘((c a))) = ((c (a) (a c)) (c a)) (REVERSE ‘((c (a) (a c)) (c a))) = ((c a) (c (a) (a c))) (CDADR ‘((c a) (c (a) (a c))) = (CDAR ‘((c (a) (a c)))) = (CDR ‘(c (a) (a c))) = ((a) (a c))	D. ((a) (a c))

19. FSAs and Regular Expressions The regular expression is: **[a-lo-r][aeiou]s?[^u-v]*** a) fortran - satisfies b) cobol - satisfies c) java - stops with the v d) lisp - satisfies e) assembly - stops with the first s f) machine - stops with the m g) python - stops with the y h) basic - satisfies i) pascal - satisfies There are 5 of them. They happen to be all high-level languages that existed prior to 1970.	C. 5
20. Assembly Language This program finds the largest triangular number that is less than 100. They are: 1, 3, 6, 10, 15, 21, 28, 36, 45, 55, 66, 78, 91, 105, ... That's 91. The formula is n(n+1)/2 = 13(13+1)/2 = 13*7 = 91 for 13 numbers, but the sequence is found by adding 2, then 3, then 4, etc..	C. 91